Computational Methods - PS1

Alastair Watt

Due February 12th

Question 1:

Using Matlab, plot on a single figure the function

$$f(x) = 1e^{-2x}, x[1, 1]$$

together with its first- and second-order Taylor approximations around the point $x = 0$. *Use a grid of 500 equally spaced nodes.*

Answer:

- $f(x) = 1 - e^{-2x}$
- $f'(x) = 2e^{-2x}$
- $f''(x) = -4e^{-2x}$

Now we plug in at $x = 0$

$$\begin{aligned} f(0) &= 1 - e^0 = 1 - 1 = 0 \\ f'(0) &= 2e^0 = 2 \\ f''(0) &= -4e^0 = -4 \end{aligned}$$

Taylor Approximations:
$T_1(x) = f(0) + f'(0)x = 0 + 2x = 2x.$
$T_2(x) = f(0) + f'(0)x + \frac{f''(0)x^2}{2!} = 0 + 2x + \frac{-4x^2}{2} = 2x - 2x^2.$
In Matlab plot $f(x)$ along with $T_1(x)$ and $T_2(x)$ over $[-1, 1]$

As expected, near $x = 0$ the Taylor polynomials closely track the true function. The linear approximation $T_1(x) = 2x$ touches $f(x)$ at $x = 0$ (as it is same value and slope) but deviates for larger and smaller x. The quadratic approximation $T_2(x) = 2x - 2x^2$ fits $f(x)$ more closely on a wider interval because it accounts for curvature (the second derivative is included) at 0. We see that for small x, the green dotted curve (second-order) almost overlaps the blue curve, which indicates a better approximation than the linear (red) curve. All curves intersect at $x = 0$ with $y = 0$, since the Taylor expansions are based around $x = 0$. This visualization confirms that including higher-order terms improves the approximation around the expansion point.

```
    % define the function f(x) and its Taylor approximations
f = @(x) 1 - exp(-2*x);     % original function f(x)
T1 = @(x) 2*x;              % first-order Taylor approx around 0
T2 = @(x) 2*x - 2*x.^2;     % second-order Taylor approx around 0

% Create 500 equally spaced points in the interval [-1, 1]
x = linspace(-1, 1, 500);

% evaluate the function and approximations on this grid
y_f  = f(x);
y_T1 = T1(x);
y_T2 = T2(x);

% plot all three on the same figure with different colours
plot(x, y_f, 'b-', 'LineWidth', 2);        % f(x) in blue solid line
hold on;
plot(x, y_T1, 'r--', 'LineWidth', 1.5);    % T1(x) in red dashed line
plot(x, y_T2, 'g:', 'LineWidth', 1.5);     % T2(x) in green dotted line
hold off;
legend('f(x) = 1 - e^{-2x}', '1st-order Taylor', '2nd-order Taylor');
xlabel('x'); ylabel('y');
title('f(x) and Taylor Approximations around x=0');
```

```
grid on;
```

MATLAB Computation

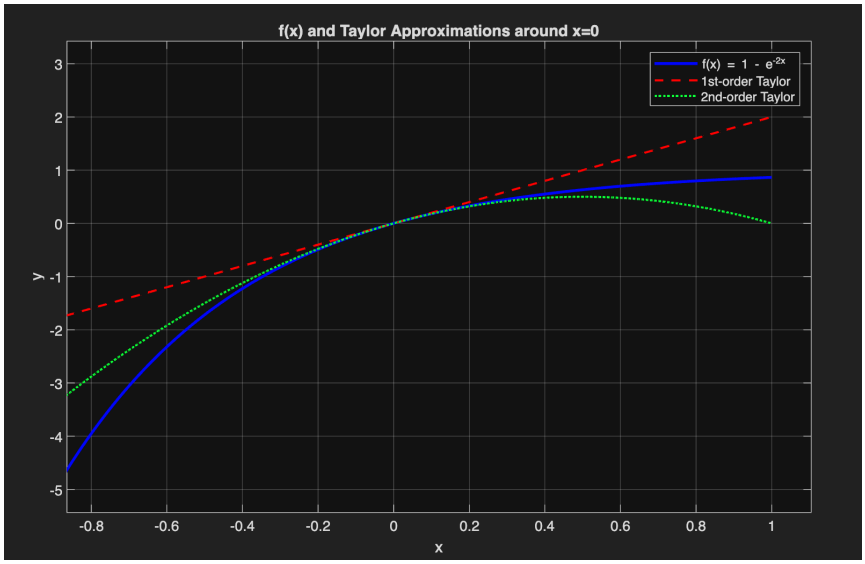

Figure 1: f(x) and Taylor approximations around x = 0

Question 2:

2.1

Solve $Ax = b$ for,

a) *Decomposition of L-U.*

b) *Gauss-Jacobi iteration.*

c) *Gauss-Seidel iteration.*

$$A = \begin{bmatrix} 54 & 14 & -11 & 2 \\ 14 & 50 & -4 & 29 \\ -11 & -4 & 55 & 22 \\ 2 & 29 & 22 & 95 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Answer:

```

% Question 2

A = [ 54  14 -11   2; % defines matrix from question
      14  50  -4  29;
     -11  -4  55  22;
       2  29  22  95];
b = [1;1;1;1]; % defines vector from question

% (a)
[L,U,P] = lu(A); % factors A out as per LU decomposition
x_LU = U \ (L \ (P*b)); % solves using factors created above

disp("x_LU =") % prints the output
disp(x_LU)

% (b)
D  = diag(diag(A));  % first diag takes into vector second makes it a matrix
R  = A - D;                  % R = L+U (everything except diagonal)

x  = zeros(4,1);             % initial guess
tol = 5e-5;                  % safe for ~4 sig figs
maxit = 1000000;     % cap on iterations

for k = 1:maxit
    x_new = D \ (b - R*x); % compute iteration

    % stop criterion, close to LU reference
    if norm(x_new - x_LU, inf) < tol
        break
    end

    x = x_new; % if not update and continue
end

x_J = x_new; % store solutions and iterations
it_J = k;
```

```
% prints the solution in a nice message
fprintf(' G a u s s Jacobi  iteration converged to the solution\n');
fprintf('[%.4f  %.4f  %.4f  %.4f]^T\n', x_J(1), x_J(2), x_J(3), x_J(4));
fprintf('after %d iterations.\n', it_J);

% does it match LU solution?
if norm(x_LU - x_J, inf) < tol
    fprintf('Jacobi solution matches LU solution to 4 significant digits.\n')
else
    fprintf('Jacobi solution does NOT match LU solution.\n')
end

% (c)

D_L = tril(A);                % D+L
U_u = triu(A,1);              % strictly upper part

for k = 1:maxit
    x_new = D_L \ (b - U_u*x); % perfrom the iteration

    if norm(x_new - x_LU, inf) < tol % same stopping rule
        break
    end

    x = x_new; % same
end

x_GS = x_new; % save answer as before
it_GS = k;

% prints message
fprintf(' G a u s s Seidel  iteration converged to the solution\n');
fprintf('[%.4f  %.4f  %.4f  %.4f]^T\n', x_GS(1), x_GS(2), x_GS(3), x_GS(4));
fprintf('after %d iterations.\n', it_GS);
```

```
% does it match LU?
if norm(x_LU - x_GS, inf) < tol
    fprintf('Seidel solution matches LU solution to 4 significant digits.\n')
else
    fprintf('Seidel solution does NOT match LU solution.\n')
end
```

MATLAB Computation

```
>> Q2_1
x_LU =
    0.0189
    0.0168
    0.0234
   -0.0004

Gauss-Jacobi iteration converged to the solution
[0.0189  0.0168  0.0233  -0.0004]^T
after 11 iterations.
Jacobi solution matches LU solution to 4 significant digits.
Gauss-Seidel iteration converged to the solution
[0.0189  0.0168  0.0234  -0.0004]^T
after 1 iterations.
Seidel solution matches LU solution to 4 significant digits.
```

Figure 2: Results from Running Code

a) L-U Decomposition

L-U decomposition factors the matrix A into smaller, simpler pieces:

$$A = P^{-1}LU$$

Where:

- L = lower triangular matrix
- U = upper triangular matrix
- P = permutation matrix

Solving $Ax = b$ becomes:

1. $PAx = Pb$

2. $LUx = Pb$

Then solve in two simple steps:

- $Ly = Pb$ (forward substitution)
- $Ux = y$ (backward substitution)

MATLAB results: `x_LU` $= \begin{bmatrix} 0.0189 & 0.0168 & 0.0234 & -0.0004 \end{bmatrix}$

(b) Gauss–Jacobi iteration

Jacobi split: $A = D + (L + U)$. Iteration:

$$x^{(k+1)} = D^{-1}(b - (A - D)x^{(k)})$$

When $R = A - D$:

$$x^{(k+1)} = D^{-1}(b - Rx^{(k)})$$

c) Gauss–Seidel iteration

Gauss–Seidel splits $A = (D + L) + U$, where:

- $D + L =$ lower triangular including diagonal
- $U =$ strictly upper triangular

Update:

$$x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)})$$

2.2

Use the MATLAB function r a n d to generate a random 100-by-100 matrix A and a random 100-vector b. Then use the MATLAB functions t i c and t o c to compute the time needed to solve the linear equation Ax = b 1, 10, and 50 times for each of the following algorithms:

a. $x = A\backslash b$.

b. $x = U\backslash(L\backslash b)$, computing the L-U factors of A only once using the MATLAB function `lu`.

c. $x = A^{-1}b$, computing A^{-1} only once using the MATLAB function `inv`.

Answer:

```

% Exercise 2.2 timing comparison

rng(0);                    % reproducible random numbers
n = 100;

A = randn(n,n);            % create random matrix
b = randn(n,1);            % create random vector

reps_list = [1 10 50];       % number of repetitions for timing

for reps = reps_list % only loop over values 1, 10 and 50

    % (a) Backslash each time
    tic
    for i = 1:reps % solve 'reps' amount of times
        x = A \ b; % method
    end
    T_a = toc; % returns elapsed seconds

    % (b) Reuse LU factorization each time
    tic
    [L,U,P] = lu(A);                % setup cost
    for i = 1:reps
        x = U \ (L \ (P*b));          % method solve cost
    end
    T_b = toc;


    % (c) Reuse inverse each time
    tic
    Ainv = inv(A);                  % setup cost
    for i = 1:reps
        x = Ainv * b;               % method solve cost
    end
    T_c = toc;

% prints results
    fprintf('\nRepetitions = %d\n', reps);
```

```
    fprintf(' (a) A\\b time        = %.6e s\n', T_a);
    fprintf(' (b) LU reuse time    = %.6e s\n', T_b);
    fprintf(' (c) inv(A)*b time    = %.6e s\n', T_c);

end
```

MATLAB Computation

```
>> Q2_2

Repetitions = 1
 (a) A\b time        = 1.905288e-03 s
 (b) LU reuse time    = 2.533702e-03 s
 (c) inv(A)*b time    = 2.799249e-02 s

Repetitions = 10
 (a) A\b time        = 6.463933e-03 s
 (b) LU reuse time    = 6.333288e-03 s
 (c) inv(A)*b time    = 1.892965e-03 s

Repetitions = 50
 (a) A\b time        = 2.268896e-02 s
 (b) LU reuse time    = 3.304678e-03 s
 (c) inv(A)*b time    = 7.490190e-04 s
>> Q2_2

Repetitions = 1
 (a) A\b time        = 2.879740e-04 s
 (b) LU reuse time    = 1.548020e-04 s
 (c) inv(A)*b time    = 3.187550e-04 s

Repetitions = 10
 (a) A\b time        = 1.515509e-03 s
 (b) LU reuse time    = 4.271840e-04 s
 (c) inv(A)*b time    = 2.762990e-04 s

Repetitions = 50
 (a) A\b time        = 8.058412e-03 s
 (b) LU reuse time    = 1.453224e-03 s
 (c) inv(A)*b time    = 4.656970e-04 s
```

Figure 3: Results from Running Above Code, Twice

To be honest I can not explain my results. (c) should naturally be the slowest in each section of repetitions while LU decomposition should improve over more repeti-

tions due to the initial setup cost being spread over a larger number of calculations. I am assuming it is due to optimization within the compiler working at different speeds or to do with general background noise on my computer.

Question 3

Consider the problem of finding the fixed point of the function $g(x) = 1 - b + bx$. *Write a Matlab script that computes the fixed point of* g, *using function iteration, and report the results for values* $b = 2, 0, 1, 2$ *starting from an initial guess* $x_0 = 0.5$.

Answer:

```
b_Values = [-2, 0, 1, 2];     % array of b values to test
x_0 = 0.5;                    % initial guess
tol = 1e-6;                  % convergence tolerance
maxIter = 1000000;           % iteration stopper

for b = b_Values
    x_old = x_0;
    fprintf('Testing b = %.1f:\n', b); % prints small header line
    for k = 1:maxIter
        % apply the iteration formula
        x_new = 1 - b + b * x_old;
        % check if the change is small enough to declare convergence
        if abs(x_new - x_old) < tol
            fprintf('  Converged to %.4f after %d iterations.\n', x_new, k);
            break;
        end
        % update x
        x_old = x_new;
    end
    % if we exited the loop without converging, report that, mostly
    % for myself during construction
    if k == maxIter
```

```
        fprintf('  Did not converge within %d iterations (last x = %.4f).\n', maxIter, x_new);
    end
end
```

MATLAB Computation

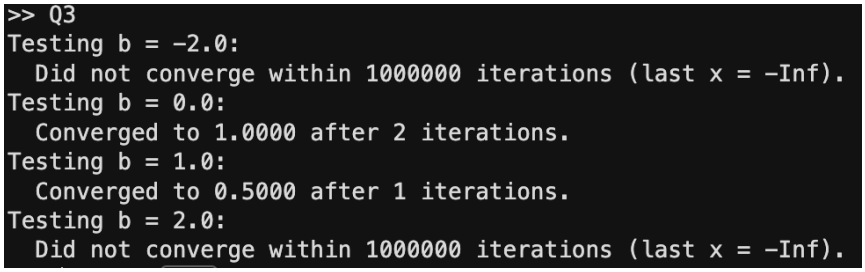

Figure 4: Results from Running Code

Fixed-point iteration: $x_{\text{new}} = 1 - b + bx_{\text{old}}$ Convergence occurs if $|b| < 1$:

- $b = 0$: Converges immediately to 1.
- $b = 1$: No contraction; does not converge.
- $b = -2, 2$: Oscillates or blows up; no convergence.

Question 4:

Write a Matlab program to find the unique root of the function $f(x) = log(x)$ by Newton's method. Include a backtracking routine so that the program works for any positive initial guess, i.e., even for those that are far away from the root of the function.

Answer:

```
function [x, k] = Q4(x0, tol, maxit)
% find the unique root of f(x)=log(x) using Newton's method with backtracking.
% Inputs
%   x0     initial guess (must be > 0)
```

```
%   tol   tolerance for |f(x)|
%   maxit maximum iterations

% Outputs:
%   x     approximate root
%   k     number of iterations used

x0 = 25
tol = 1e-6
maxit = 100000

    if x0 <= 0 % domain check needed for log
        error('Initial guess x0 must be positive.');
    end

    % function and derivative
    f  = @(x) log(x);
    fp = @(x) 1./x;

    % merit function
    phi = @(x) 0.5*(f(x)^2);

    % Backtracking parameters
    rho = 0.5;     % shrink factor
    x = x0;

    for k = 1:maxit

        fx = f(x);

        % Convergence test
        if abs(fx) < tol
            fprintf('Root = %.1f:\n',x)
            fprintf('Number of iterations = %.1f\n',k)
            return
        end

        % Newton direction
        p = -fx / fp(x);

        % Backtracking line search
```

```
Root = 1.0:
Number of iterations = 5.0

ans =

    1.0000
```

Figure 5: Results from Above Code

```
        alpha = 1.0;

        % keep shrinking alpha until:
        % (1) next x stays in domain: x + alpha*p > 0
        % (2) merit function decreases: phi(next) < phi(current)
        while (x + alpha*p) <= 0 || phi(x + alpha*p) >= phi(x)
            alpha = rho * alpha;

            % safety stop (prevents infinite loop) and for myself when
            % coding it
            if alpha < 1e-16
                error('Backtracking failed: step size became too small.');
            end
        end

        % Accept step
        x = x + alpha*p;
    end

    error('Newton method did not converge within maxit iterations.');
end
```

MATLAB Computation

Rootfinding for $f(x) = \log(x)$ using Newton's method with backtracking line search. The algorithm ensures:

- $x > 0$ (staying within the domain).
- The step decreases the merit function $\phi(x) = \frac{1}{2}f(x)^2$.

The algorithm converges toward the true root $x = 1$.

Question 5:

***I ran into a lot of difficulties with Q5 so apologies if this code is more complex than it needs to be. I ran into many issues especially in Ex3.3 but I think I got some form of coherent results.

Exercise 3.3

The Black-Scholes option pricing formula expresses the value of an option as a function of the current value S of the underlying asset, the option's strike price K, the time to maturity τ, the current risk-free interest rate r, a dividend rate δ, and the volatility of the price of the underlying asset σ. The formula for a call option is:

$$V(S, K, \tau, r, \delta, \sigma) = e^{-\delta\tau} S\Phi(d) - e^{-r\tau} K\Phi(d - \sigma\sqrt{\tau})$$

where

$$d = \frac{\ln(e^{-\delta\tau}S) - \ln(e^{-r\tau}K)}{\sigma\sqrt{\tau}} + \frac{1}{2}\sigma\sqrt{\tau}$$

and Φ is the standard normal cumulative distribution function (CDF):

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\frac{1}{2}z^2} dz$$

a. *Write a MATLAB procedure that takes the six inputs and returns the Black-Scholes option value:* `V = BSVal(S, K, tau, r, delta, sigma)`. *Use CompEcon Toolbox function* `cdfn` *to compute the standard normal CDF.*

b. *All of the inputs to the Black-Scholes formula are readily observable except σ. Market participants often want to determine the value of σ implied by the market price of an option. Write a program that computes the so-called implied volatility with the calling syntax:* `sigma = ImpVol(S, K, tau, r, delta, V)`.

The algorithm should use Newton's method to solve (for σ) the rootfinding problem $V - BSVal(S, K, \tau, r, \delta, \sigma) = 0$. To do so, use the derivative of the Black-Scholes formula with respect to σ, which equals:

$$\frac{\partial V}{\partial \sigma} = Se^{-\delta\tau}\sqrt{\tau/(2\pi)}e^{-0.5d^2}$$

The program should be stand-alone and not call any rootfinding solver or numerical derivative algorithm, though it may call ***`BSVal`*** *from part a.*

Answer:

```
% sourced from question

delta = 0; % continuous dividend yield
tau   = 1; % time to maturity in years
K     = 1.1; % strike
S     = 1; % spot price today
r     = 0.08; % continuous compunded risk-free interest rate
V_mkt = 0.0728; % the observed market option price

[sigma_hat, it] = ImpVol(S, K, tau, r, delta, V_mkt); % calls function where sigma_hat is the implied
% volatality estimate and it is number of newton iterations

fprintf("Implied volatility = %.12f (iterations = %d)\n", sigma_hat, it);
fprintf("Check price: BSVa1 = %.10f\n", BSVa1(S,K,tau,r,delta, sigma_hat)); % just to confirm it matches


function [sigma, it] = ImpVol(S, K, tau, r, delta, V_mkt, sigma0, tol, maxIt)
%ImpVol Implied volatility for a call using Newton's method.

% Required inputs are above
%   S, K, tau, r, delta, V_mkt

% Can change these below when needed
%   sigma0 initial guess
%   tol    tolerance on |V_model - V_mkt| (default 1e-10)
```

```
%   maxIt  max iterations (default 100)

% Outputs:
%   sigma implied vol
%   it    iterations used

% just to have a fail safe if optional inputs arent provided
    if nargin < 7 || isempty(sigma0), sigma0 = 1; end
    if nargin < 8 || isempty(tol),    tol    = 1e-10; end
    if nargin < 9 || isempty(maxIt),  maxIt  = 100; end

    sigma = sigma0; % start iterations at given sigma

    for it = 1:maxIt
        V_model = BSVa1(S, K, tau, r, delta, sigma);
        f = V_model - V_mkt; % define root finding function

        if abs(f) < tol % if within tolerance stop
            fprintf("Converged: sigma = %.12f after %d iterations\n", sigma, it);
            return
        end

        sqrtT = sqrt(tau); % square root tau

        % d as defined in question
        d = ( log(exp(-delta*tau)*S) - log(exp(-r*tau)*K) )/(sigma*sqrtT) ...
            + 0.5*sigma*sqrtT;

        % V partial defined from question
        vega = S*exp(-delta*tau)*sqrt(tau/(2*pi))*exp(-0.5*d^2);

        if abs(vega) < 1e-14 % just to protect against divison by ~0
            error("Vega too small; Newton update unstable. Try a different sigma0.");
        end

        sigma = sigma - f/vega; % newton step
```

```
        % enforce sigma > 0
        if sigma <= 0
            sigma = 1e-8;
        end
    end
% if did not converge display
    error("Newton did not converge in %d iterations (last sigma=%.6g).", maxIt, sigma);
end


function V = BSVa1(S, K, tau, r, delta, sigma)
%BSVa1  B l a c k Scholes  call option value using CompEcon cdfn for N(0,1) CDF
%as instructed

% Inputs
%   S     - spot price
%   K     - strike
%   tau   - time to maturity
%   r     - risk-free rate
%   delta - continuous dividend yield
%   sigma - volatility

% Output
%   V     call option value

    if tau <= 0 % ensuring positivity
        error("tau must be > 0");
    end
    if sigma <= 0
        error("sigma must be > 0");
    end

    sqrtT = sqrt(tau);

    % d again
    d = ( log(exp(-delta*tau)*S) - log(exp(-r*tau)*K) )/(sigma*sqrtT) ...
        + 0.5*sigma*sqrtT;
```

```
    % Use CompEcon Toolbox standard normal CDF
    Phi_d  = cdfn(d);
    Phi_d2 = cdfn(d - sigma*sqrtT);

    V = exp(-delta*tau)*S*Phi_d - exp(-r*tau)*K*Phi_d2; % exactly as defined in question
    % in order to check call option value
end
```

MATLAB Computation

```
>> Q5_Ex33
Converged: sigma = 0.200023976924 after 4 iterations
Implied volatility = 0.200023976924 (iterations = 4)
Check price: BSVa1 = 0.0728000000
```

Figure 6: Results from Running Above Code

Exercise 3.5

Consider the function $f : \mathbb{R}^2 \mapsto \mathbb{R}^2$ *defined by:*

$$f_1(x) = 200x_1(x_2 - x_1^2) - x_1 + 1$$
$$f_2(x) = 100(x_1^2 - x_2)$$

Write a MATLAB function that takes a column 2-vector x *as input and returns* `fval`*, a column 2-vector containing the value of* f *at* x*, and* `fjac`*, a 2-by-2 matrix containing the Jacobian of* f *at* x*.*

a. *Compute numerically the root of* f *using Newton's method.*

b. *Compute numerically the root of* f *using Broyden's method.*

Answer:

```

function Q5_Ex35()
% Exercise 3.5
% Newton's method + Broyden's method for solving f(x)=0 in R^2
```

```

clc; clear;


x0       = [-0.5; 1.0];    % start
tolF     = 1e-10;          % tolerance for ||f(x)||
tolX     = 1e-12;          % tolerance for ||step||
maxItNewton = 100;
maxItBroyden = 1000;    % Broyden often needed more iterations

fprintf('Target root should be (1,1).\n\n');

% ----------------------------
% (a) Newton
% ----------------------------
fprintf('=== Newton''s Method ===\n');
[xN, fN, itN] = newton_solver(x0, tolF, tolX, maxItNewton);
fprintf('Newton finished in %d iterations.\n', itN);
fprintf('x = [% .15e; % .15e]\n', xN(1), xN(2));
fprintf('||f(x)|| = %.3e\n\n', norm(fN));

% ----------------------------
% (b) Broyden
% ----------------------------
fprintf('=== Broyden''s Method (with safeguard for tiny steps)
    ===\n');
[xB, fB, itB] = broyden_solver(x0, tolF, tolX, maxItBroyden);
fprintf('Broyden finished in %d iterations.\n', itB);
fprintf('x = [% .15e; % .15e]\n', xB(1), xB(2));
fprintf('||f(x)|| = %.3e\n', norm(fB));

end

% ==========================================================
% Function + Jacobian
% ==========================================================
function [fval, fjac] = f_and_jac(x)
x1 = x(1);
x2 = x(2);

f1 = 200*x1*(x2 - x1^2) - x1 + 1; % as given in question
```

```
f2 = 100*(x1^2 - x2);

fval = [f1; f2];

df1dx1 = 200*x2 - 600*x1^2 - 1; % derivatives
df1dx2 = 200*x1;
df2dx1 = 200*x1;
df2dx2 = -100;

fjac = [df1dx1, df1dx2; % define jacobian
        df2dx1, df2dx2];
end

% ==========================================================
% Newton solver
% ==========================================================
function [x, fval, it] = newton_solver(x0, tolF, tolX, maxIt)

x = x0;

for it = 1:maxIt
    [fval, J] = f_and_jac(x);

    fnorm = norm(fval);
    if fnorm < tolF % tolerance for function
        return;
    end

    % Solve J*s = -f (more stable than inv(J))
    s = -J \ fval;

    if norm(s) < tolX % tolerance for step
        return;
    end

    x = x + s;

    printEvery = 3;   % prints every 3 iterations to check behaviour
    if mod(it, printEvery) == 0
    fprintf('it=%3d  ||f||=%.3e  ||s||=%.3e  x=[% .6e,% .6e]\n
```

```
        ', ...
         it, fnorm, norm(s), x(1), x(2));
    end

end
end

% ==========================================================
% Broyden solver, more in depth as I encountered many problems
% Safeguards
%   - if step is tiny, denominator s'*s is tiny -> unstable update
%   - if B becomes poorly conditioned, reset B to true Jacobian
%   - optional damping if needed
% ==========================================================
function [x, fval, it] = broyden_solver(x0, tolF, tolX, maxIt)

x = x0;
[fval, Jtrue] = f_and_jac(x);

% start with true Jacobian at x0
B = Jtrue;

% safeguards + controls
stepDenomTol = 1e-14;     % for p'*p
condTol      = 1e12;      % reset if ill-conditioned
useDamping   = true;      % turns on and off damping

printEvery   = 20;        % set Inf for silent

for it = 1:maxIt
    fnorm = norm(fval);
    if fnorm < tolF
        return;
    end

    % Direction from current B
    s = -B \ fval;

    if norm(s) < tolX
```

```
        return;
    end

    % line search on merit phi = 1/2 ||f||^2
    alpha = 1.0;
    phi0  = 0.5 * (fnorm^2); % define the merit function

    p = alpha * s; % alter with alpha
    x_new = x + p;
    [f_new, ~] = f_and_jac(x_new);
    phi_new = 0.5 * (norm(f_new)^2);

    p = x_new - x;          % actual step taken

    if useDamping
        c1 = 1e-4;
        % approximate directional derivative of phi
        % if the step doesnt reduce phi halve alpha and retry
        while phi_new > (1 - 2*c1*alpha) * phi0 && alpha > 1e-10
            alpha = 0.5 * alpha;
            p = alpha * s;
            x_new = x + p;
            [f_new, ~] = f_and_jac(x_new);
            phi_new = 0.5 * (norm(f_new)^2);
        end
    end


    p = x_new - x; % recompute actual step

    y = f_new - fval; % must match actual step

    denom = (p' * p);

    % If damping had to shrink alpha to ~ zero, the direction from B is unhelpful.
    % Reset B to the true Jacobian at x_new so the next iteration is well-posed.
    % had to include these as I was running into problems (might be cheating)
```

```
    if alpha <= 1e-10
        [~, Jreset] = f_and_jac(x_new);
        B = Jreset;
    end

    % if step too small, reset B
    if denom < stepDenomTol
        [~, Jreset] = f_and_jac(x_new);
        B = Jreset;
    else % otherwise
        Bp = B * p;
        B  = B + ((y - Bp) * (p')) / denom;   % good Broyden update using p
    end

    % if B is not nice, reset
    if rcond(B) < 1/condTol
        [~, Jreset] = f_and_jac(x_new);
        B = Jreset;
    end

    % Accept step
    x = x_new;
    fval = f_new;

    if mod(it, printEvery) == 0
        fprintf('it=%3d  ||f||=%.3e  ||p||=%.3e  alpha=%.2e  x=[% .6e,% .6e]\n', ...
            it, fnorm, norm(p), alpha, x(1), x(2));
    end
end
end
```

MATLAB Computation

```
Target root should be (1,1).

=== Newton's Method ===
it=  3  ||f||=4.781e+02  ||s||=2.191e+00  x=[ 9.700683e-01, 9.410325e-01]
it=  6  ||f||=4.246e-09  ||s||=9.494e-09  x=[ 1.000000e+00, 1.000000e+00]
Newton finished in 7 iterations.
x = [ 1.000000000000000e+00;  1.000000000000000e+00]
||f(x)|| = 0.000e+00

=== Broyden's Method (with safeguard for tiny steps) ===
it= 20  ||f||=1.503e+00  ||p||=5.712e-03  alpha=3.91e-03  x=[-4.279650e-01,
it= 40  ||f||=1.477e+00  ||p||=9.686e-03  alpha=1.56e-02  x=[-2.894986e-01,
it= 60  ||f||=1.056e+00  ||p||=2.909e-02  alpha=5.00e-01  x=[ 9.578069e-02,
it= 80  ||f||=2.913e-01  ||p||=2.898e-02  alpha=6.25e-02  x=[ 5.810850e-01,
it=100  ||f||=2.520e-01  ||p||=9.067e-03  alpha=3.12e-02  x=[ 7.099357e-01,
it=120  ||f||=2.020e-01  ||p||=1.126e-02  alpha=6.25e-02  x=[ 8.215134e-01,
it=140  ||f||=1.359e-01  ||p||=6.931e-03  alpha=6.25e-02  x=[ 9.337367e-01,
Broyden finished in 154 iterations.
x = [ 1.000000000088154e+00;  1.000000000176537e+00]
||f(x)|| = 4.816e-11
```

Figure 7: Results from Running Above Code

Exercise 3.7

Consider a simple endowment economy with three agents and two goods. Agent i is initially endowed with e_{ij} units of good j and maximizes utility:

$$U_i(x) = \sum_{j=1}^{2} a_{ij}(v_{ij}+1)^{-1} x_{ij}^{v_{ij}+1}$$

subject to the budget constraint:

$$\sum_{j=1}^{2} p_j x_{ij} = \sum_{j=1}^{2} p_j e_{ij}$$

where x_{ij} is the amount of good j consumed by agent i, p_j is the market price of good j, and $a_{ij} > 0$ and $v_{ij} < 0$ are preference parameters. A competitive general equilibrium for the endowment economy si apair of relative prices, p1 and p2, normalized to sum to one, such that all the goods markets clear if each agent maximizes utility subject to his budget constraints. Compute the competitive general equilibrium (relative prices p_1 and p_2 normalized to sum to one) such that all markets clear for the following parameters:

(i, j)	a_{ij}	v_{ij}	e_{ij}
(1, 1)	2.0	-2.0	2.0
(1, 2)	1.5	-0.5	3.0
(2, 1)	1.5	-1.5	1.0
(2, 2)	2.0	-0.5	2.0
(3, 1)	1.5	-0.5	4.0
(3, 2)	2.0	-1.5	0.0

Answer:

```
function Q5_Ex37()
% Exercise 3.7
% 3 agents, 2 goods need t solve equilibrium prices p=[p1;p2]
   with normalization p1+p2=1
% using damped Newton with a finite-difference Jacobian.

% robustness features
%   - Inner lambda_i solved by bisection
%   - Damping/backtracking keeps prices positive and reduces
   ||F||.
%   - Finite-difference Jacobian avoids algebra mistakes

clc; clear;

% ---------------------------------------------------------------
% Parameters from the table (i=1..3, j=1..2)
% ---------------------------------------------------------------
a = [2.0, 1.5; % agent 1 has a_11 = 2.0 and a_12 = 1.5 etc
     1.5, 2.0;
     1.5, 2.0];

v = [-2.0, -0.5;
     -1.5, -0.5;
     -0.5, -1.5];

e = [2.0, 3.0;
     1.0, 2.0;
     4.0, 0.0];

E = sum(e,1)';   % total endowment by good: [E1; E2]
```

```
fprintf('Total endowments: E1=%.1f, E2=%.1f\n', E(1), E(2));

% ------------------------------------------------------------
% Initial guess for prices (must be positive and sum to 1)
% ------------------------------------------------------------
p = [0.5; 0.5];

% Tolerances / iteration cap
tolF  = 1e-10;     % stop when ||F|| < tolF
tolX  = 1e-12;     % or when ||step|| < tolX
maxIt = 200;        % classic max iterations

% ------------------------------------------------------------
% Damped/altered Newton iterations
% ------------------------------------------------------------
for it = 1:maxIt
    F = equilibrium_F(p, a, v, e, E); % complicated output
    Fn = norm(F);

    if Fn < tolF
        break;
    end

    J = fd_jacobian(@(pp) equilibrium_F(pp, a, v, e, E), p); %
        jacobian of F

    % classic newton step
    s = -J \ F;

    if norm(s) < tolX % step-size criteria
        break;
    end

    % backtracking line search to keep p>0 and decrease ||F||
    alpha = 1.0;
    while alpha > 1e-8
        p_new = p + alpha*s;

        if all(p_new > 0)
            F_new = equilibrium_F(p_new, a, v, e, E);
```

```
            if norm(F_new) <= (1 - 1e-4*alpha)*Fn % shrinks alpha until step actually improves
                break;
            end
        end

        alpha = 0.5*alpha;
    end % crucial segment to ensure we are always actually going somewhere

    p = p + alpha*s;
    % helpful to keep track of iterations, mostly for myself to actually
    % see in action
    fprintf('it=%3d  ||F||=%.3e  ||s||=%.3e  alpha=%.2e  p=[%.12f, %.12f]\n', ...
        it, Fn, norm(s), alpha, p(1), p(2));
end

% ------------------------------------------------------------
% Report results
% ------------------------------------------------------------
F = equilibrium_F(p, a, v, e, E);
fprintf('\n=== Solution ===\n');
fprintf('p1 = %.12f\n', p(1));
fprintf('p2 = %.12f\n', p(2));
fprintf('p1+p2 = %.12f\n', p(1)+p(2)); % should always equal to 1 for normalisation
fprintf('||F|| = %.3e\n\n', norm(F));

print_allocations(p, a, v, e, E);

end

% ============================================================
% equilibrium system F(p) = [Z1(p); p1+p2-1]
% ============================================================
function F = equilibrium_F(p, a, v, e, E)
p1 = p(1); p2 = p(2); % extract scalars

% guard against invalid prices
```

```
if p1 <= 0 || p2 <= 0
    F = [1e6; 1e6];
    return;
end

Xtot = [0;0]; % total demand
for i = 1:3
    xi = agent_demand(i, p, a, v, e);  % 2x1
    Xtot = Xtot + xi;
end

% just one clearing and normalisation condition needed
Z1     = Xtot(1) - E(1);   % excess demand good 1
normEq = p1 + p2 - 1;      % normalization

F = [Z1; normEq];
end

% ============================================================
% probably easier ways to do this but this is what I cam to
% agent i demand given prices p = [p1;p2]
% ============================================================
function xi = agent_demand(i, p, a, v, e)
p1 = p(1); p2 = p(2);
m  = p1*e(i,1) + p2*e(i,2);  % income is value of endowment at
    prices p

% Budget residual in lambda
% spending(lambda) - income, from Lagrangian
budget_res = @(lam) ...
    p1*( (lam*p1/a(i,1))^(1/v(i,1)) ) + ...
    p2*( (lam*p2/a(i,2))^(1/v(i,2)) ) - m;

% Bracket for lambda needs to be wide, (0, +inf)
% For very small lambda, 1/v < 0 makes x large => spending
   large => residual > 0.
% For very large lambda, x tiny => spending tiny => residual <
    0.
lo = 1e-12; hi = 1e12;

flo = budget_res(lo);
```

```
fhi = budget_res(hi);

% widen and repair bracket if needed
if flo < 0
    lo = 1e-30;
    flo = budget_res(lo);
end
if fhi > 0
    hi = 1e30;
    fhi = budget_res(hi);
end

% Bisection
lam = sqrt(lo*hi);
for k = 1:250
    lam = sqrt(lo*hi);            % geometric midpoint
    fmid = budget_res(lam);      % hold midpoint

    if abs(fmid) < 1e-12       % stopping condition
        break;
    end

    if fmid > 0 % update if needs be
        lo = lam;
    else
        hi = lam;
    end
end

x1 = (lam*p1/a(i,1))^(1/v(i,1)); % where lambda was chosen to
   satisfy budget
x2 = (lam*p2/a(i,2))^(1/v(i,2));
xi = [x1; x2];
end

% ============================================================
% Finite-difference Jacobian (central differences)
% ============================================================
function J = fd_jacobian(Ffun, x)
n  = length(x);
fx = Ffun(x);
```

```
m  = length(fx);

J = zeros(m,n); % here n = 2 and m = 2

h_base = 1e-6; % makes relative when x(k) is big
for k = 1:n
    h = h_base * max(1, abs(x(k)));
    xp = x; xm = x;
    xp(k) = xp(k) + h;
    xm(k) = xm(k) - h;
    J(:,k) = (Ffun(xp) - Ffun(xm)) / (2*h); % approximates
end
end

% ==============================================================
% Print equilibrium allocations and market clearing check
% ==============================================================
function print_allocations(p, a, v, e, E)
fprintf('--- Allocations at p=[%.12f, %.12f] ---\n', p(1), p
   (2));
Xtot = [0;0];
for i = 1:3
    xi = agent_demand(i, p, a, v, e);
    Xtot = Xtot + xi;
    fprintf('Agent %d: x1=%.10f, x2=%.10f\n', i, xi(1), xi(2))
       ;
end
fprintf('Totals: X1=%.10f (E1=%.1f), X2=%.10f (E2=%.1f)\n',
   ...
    Xtot(1), E(1), Xtot(2), E(2));
fprintf('Excess: Z1=%.3e, Z2=%.3e (Z2 implied by Walras/
   normalization)\n', ...
    Xtot(1)-E(1), Xtot(2)-E(2));
end
```

MATLAB Computation

```
Total endowments: E1=7.0, E2=5.0
it=  1  ||F||=2.013e+00  ||s||=8.755e-01  alpha=5.00e-01  p=[0.1904642292
it=  2  ||F||=2.883e-01  ||s||=2.365e-02  alpha=1.00e+00  p=[0.2071863990
it=  3  ||F||=1.974e-02  ||s||=1.865e-03  alpha=1.00e+00  p=[0.2085054275
it=  4  ||F||=1.053e-04  ||s||=1.005e-05  alpha=1.00e+00  p=[0.2085125361
it=  5  ||F||=3.026e-09  ||s||=2.891e-10  alpha=1.00e+00  p=[0.2085125363

=== Solution ===
p1 = 0.208512536352
p2 = 0.791487463648
p1+p2 = 1.000000000000
||F|| = 1.963e-13

--- Allocations at p=[0.208512536352, 0.791487463648] ---
Agent 1: x1=2.9004285883, x2=2.7627875900
Agent 2: x1=2.3685507147, x2=1.6394636760
Agent 3: x1=1.7310206970, x2=0.5977487340
Totals: X1=7.0000000000 (E1=7.0), X2=5.0000000000 (E2=5.0)
Excess: Z1=-1.963e-13, Z2=-8.828e-13 (Z2 implied by Walras/normalization)
```

Figure 8: Results from Above Code